context
context

AIEBuilder

builders : dict

__init__()
build(model: nengo.builder.Model, obj: nengo.base.NengoObject)

AIEConnection

_to_neurons
function
has_weights
is_decoded
learning_rule
post_obj
post_slice
pre_obj
pre_slice
size_in
size_mid
size_out

__init__(pre: nengo.Ensemble | nengo.ensemble.Neurons | nengo.Node, post: nengo.Ensemble | nengo.ensemble.Neurons | nengo.Node | nengo.connection.LearningRule, synapse: nengo.synapses.Synapse | nengo.params.DefaultType | None, function: collections.abc.Callable | numpy.ndarray | nengo.params.DefaultType, transform: numpy.ndarray | nengo.params.DefaultType, solver: nengo.solvers.Solver | nengo.params.DefaultType, learning_rule_type: nengo.learning_rules.LearningRuleType | collections.abc.Iterable[nengo.learning_rules.LearningRuleType] | nengo.params.DefaultType, eval_points: numpy.ndarray | int | nengo.params.DefaultType, scale_eval_points: bool | nengo.params.DefaultType, label: str | nengo.params.DefaultType, seed: str | nengo.params.DefaultType)

AIEElementwiseInc

A
X
Y
_descstr
context
input_bo
output_bo
size

__init__(A: nengo.builder.Signal, X: nengo.builder.Signal, Y: nengo.builder.Signal, tag)
make_step(signals: dict, dt: float, rng): Callable

AIELif

context
dt : float
input_bo
output_bo
size

__init__(size: int, dt: float, tau_rc: float, tau_ref: float, min_voltage: float, amplitude: float, initial_state)
step(dt: float, J: npt.NDArray, output: npt.NDArray, voltage: npt.NDArray, refractory_time: npt.NDArray)

AIEContext

__bos : dict
__device
__instr_bo
__instr_v : list[int]
__kernel
__next_gid : int

__init__(device: xrt.device, kernel: xrt.kernel, instr_v: list[int], instr_bo: xrt.bo)
create_inout_bo(name: str, volume: int, itemsize: int): xrt.bo
get_bo(name: str): xrt.bo
get_device(): xrt.device
get_kernel(): xrt.kernel
kernel_call()

AIEDense

_argreprs
init_shape
size_in
size_out

__init__(shape: tuple, init: float)

AIEManager

init_aie(xclbin_path): tuple[xrt.device, xrt.kernel]
load_insts(insts_path, device, kernel): tuple[list[int], xrt.bo]
next_multiple_of(divider)

AIESimulator

__init__(network: nengo.Network, dt: float | None, seed: int | None, progress_bar: bool | ProgressBar | None)

MlirBuilder

__insts_path
__mlir_source
__xclbin_path
mlir_source

__init__()
build(device: Resolvable, size: int): tuple[str, str]
compile(): tuple[str, str]

ElementwiseIncBuilder

__init__()
build(device: Resolvable, size: int): tuple[str, str]

LIFNeuronBuilder

__init__()
build(device: Resolvable, size: int): tuple[str, str]

KernelManager

__kernel_objects : dict
__kernel_sources : dict
resources

compile_all()
get_kernel_object_for(opname: OpNames): str
register_kernel_source(opname: OpNames)

OpNames

name